

SISTEMAS NUMÈRICOS

Ing. Edgar Andrés Gutiérrez Cáceres

SISTEMAS NUMÉRICOS

Estás acostumbrado a trabajar con números decimales.

En los sistemas digitales, compuestos por 1 y 0, los números binarios o hexadecimales suelen ser más convenientes.

Esta sección presenta los distintos sistemas numéricos que se utilizarán a lo largo del diseño de sistemas digitales.

NUMEROS DECIMALES

En la escuela primaria, aprendiste a contar y a hacer operaciones aritméticas con decimales. Así como (probablemente) tienes diez dedos, existen diez dígitos decimales: 0, 1, 2, ..., 9. Los dígitos decimales se unen para formar números decimales más largos.

Cada columna de un número decimal tiene diez veces el peso de la columna anterior. De derecha a izquierda, los pesos de las columnas son 1, 10, 100, 1000, y así sucesivamente.

Los números decimales se denominan de base 10. La base se indica con un subíndice después del número para evitar confusiones al trabajar con más de una base.

NUMEROS DECIMALES

Este ejemplo muestra cómo se escribe el número decimal $9742_{(10)}$ como la suma de cada uno de sus dígitos multiplicada por el peso de la columna correspondiente.

1's column
10's column
100's column
1000's column

$$9742_{10} = 9 \times 10^3 + 7 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

 nine seven four two
 thousands hundreds tens ones

NUMEROS DECIMALES

Un número decimal de N dígitos representa una de 10^N posibilidades: 0, 1, 2, 3, ..., $10^N - 1$.

Esto se denomina rango del número.

Por ejemplo, un número decimal de tres dígitos representa una de 1000 posibilidades en el rango de 0 a 999.

NUMEROS BINARIOS

Los bits representan uno de dos valores, 0 o 1, y se combinan para formar números binarios.

Cada columna de un número binario tiene el doble de peso que la anterior, por lo que los números binarios son de base 2.

En binario, los pesos de las columnas (de derecha a izquierda) son 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, y así sucesivamente.

Si trabajas frecuentemente con números binarios, ahorrarás tiempo si recuerdas estas potencias de dos hasta 2^{16} .

NUMEROS BINARIOS

Un número binario de N bits representa una de las 2^N posibilidades: 0, 1, 2, 3, ..., $2^N - 1$.

La tabla muestra los números binarios de 1, 2, 3 y 4 bits y sus equivalentes decimales.

1-Bit Binary Numbers	2-Bit Binary Numbers	3-Bit Binary Numbers	4-Bit Binary Numbers	Decimal Equivalents
0	00	000	0000	0
1	01	001	0001	1
	10	010	0010	2
	11	011	0011	3
		100	0100	4
		101	0101	5
		110	0110	6
		111	0111	7
			1000	8
			1001	9
			1010	10
			1011	11
			1100	12
			1101	13
			1110	14
			1111	15

NUMEROS BINARIOS - EJEMPLO

Convierta el numero binario 10110_2 a decimal

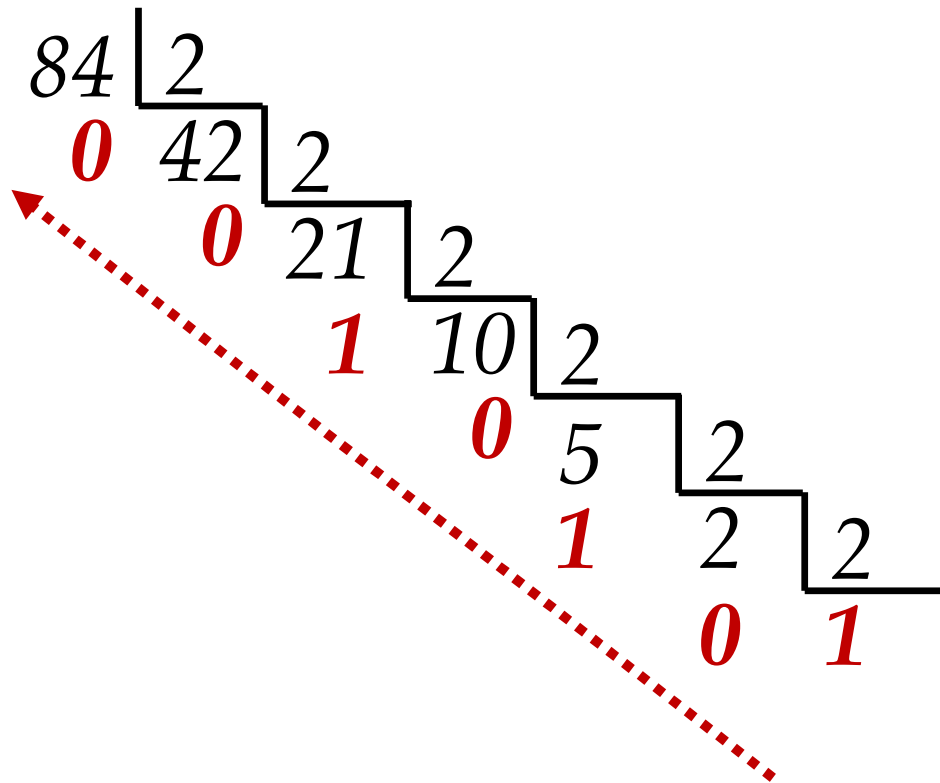
1's column
2's column
4's column
8's column
16's column

$$10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$$

one sixteen no eight one four one two no one

NUMEROS BINARIOS - EJEMPLO

Convierta el numero decimal 84_{10} a binario



.....
 1010100_2

NUMEROS BINARIOS - EJEMPLO

Convierta el numero decimal 84_{10} a binario

2^6	2^5	2^4	2^3	2^2	2^1	2^0	
64	32	16	8	4	2	1	
64		16		4			= 84
1	0	1	0	1	0	0	= 84

NUMEROS HEXADECIMALES

Escribir números binarios largos se vuelve tedioso y propenso a errores.

Un grupo de cuatro bits representa una de $2^4 = 16$ posibilidades.

Por lo tanto, a veces es más conveniente trabajar en base 16, también conocida como hexadecimal.

NUMEROS HEXADECIMALES

Los números hexadecimales utilizan los dígitos del 0 al 9 junto con las letras de la “A” a la “F”, como se muestra en la Tabla.

Las columnas en base 16 tienen pesos de 1, 16, 16^2 (o 256), 16^3 (o 4096), etc.

Hexadecimal Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

NUMEROS HEXADECIMALES

Convierta el numero hexadecimal $2ED_{16}$ a binario y a decimal

2 **E** **D**

0010₂ **1110₂** **1101₂**

001011101101₂

Hexadecimal Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

NUMEROS HEXADECIMALES - EJEMPLO

Convierta el numero hexadecimal $2ED_{16}$ a binario y a decimal

1's column
16's column
256's column

$$2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$$

two
two hundred
fifty six's fourteen
sixteens thirteen
ones

NUMEROS HEXADECIMALES - EJEMPLO

Convierta el numero hexadecimal $2ED_{16}$ a binario y a decimal

2×16^2	+	$E \times 16^1$	+	$D \times 16^0$	=	749_{10}
2×16^2	+	14×16^1	+	13×16^0	=	749_{10}
2×256	+	14×16	+	13×1	=	749_{10}
512	+	224	+	13	=	749_{10}

NUMEROS HEXADECIMALES

*Convierta el numero binario
1111010₂ a hexadecimal*

1	1	1	1	0	1	0
7			A			

7A₁₆

Hexadecimal Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

NUMEROS HEXADECIMALES - EJEMPLO

Convierta el numero decimal 333₁₀ a hexadecimal y binario

2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	=									
256	128	64	32	16	8	4	2	1	=									
256	+	0	+	64	+	0	+	0	+	8	+	4	+	0	+	1	=	333 ₁₀
1		0		1		0		0		1		1		0		1	=	333 ₁₀
1	4				D										=	333 ₁₀		

101001101₁₆

14D₁₆

Bytes, Nibbles

Un grupo de ocho bits se llama **byte**. Representa una de $2^8 = 256$ posibilidades.

El tamaño de los objetos almacenados en las memorias de las computadoras se mide habitualmente en bytes en lugar de bits.

Un grupo de cuatro bits, o medio byte, se denomina **nibble**. Representa una de $2^4 = 16$ posibilidades.

Un dígito hexadecimal almacena un cuarteto y dos dígitos hexadecimales almacenan un byte completo. Los nibble ya no son una unidad de uso común, pero el término es a veces mencionado.

Bytes, Nibbles

*Los microprocesadores manejan datos en fragmentos llamados **words**. El tamaño de una palabra depende de la arquitectura del microprocesador.*

La mayoría de las computadoras tienen procesadores de 64 bits, lo que indica que funcionan con words de 64 bits.

Se encuentran ampliamente disponibles ordenadores más antiguos que manejan words de 32 bits.

Los microprocesadores más simples, especialmente son utilizados en aparatos como tostadoras, utilizan words de 8 o 16 bits.

Bytes, Nibbles

Dentro de un grupo de bits, el bit en la columna de unos se llama **bit menos significativo (lsb)** y el bit en el otro extremo se llama **bit más significativo (msb)**, como se muestra en la Figura para un número binario de 6 bits.

101100

most least

significant significant

bit bit

Bytes, Nibbles

De manera similar, dentro de una palabra, los bytes se identifican como el byte menos significativo (LSB) hasta el byte más significativo (MSB), como se muestra en la Figura para un número de 4 bytes escrito con ocho dígitos hexadecimales.

DEAFDAD8

most significant byte least significant byte

Bytes, Nibbles

Por cómoda coincidencia, $2^{10} = 1024 \approx 10^3$. Por lo tanto, el término kilo (mil en griego) indica 2^{10} .

Por ejemplo, 2^{10} bytes son un kilobyte (1 KB). De manera similar, mega (millón) indica $2^{20} \approx 10^6$ y giga (mil millones) indica $2^{30} \approx 10^9$.

Si conoces $2^{10} \approx 1$ mil, $2^{20} \approx 1$ millón, $2^{30} \approx 1$ billón, y recuerdas las potencias de dos hasta 29, es fácil estimar cualquier potencia de dos en tu cabeza.

Bytes, Nibbles - Ejemplo

Encuentra el valor aproximado de 2^{24} sin usar una calculadora.

Solución:

Divide el exponente en un múltiplo de diez y el resto.

$$2^{24} = 2^{20} \times 2^4.$$

$2^{20} \approx 1$ millón. $2^4 = 16$. Entonces, $2^{24} \approx 16$ millones.

Técnicamente, $2^{24} = 16.777.216$, pero 16 millones es lo suficientemente cercano para fines de marketing.

Bytes, Nibbles

1024 bytes se denomina kilobyte (KB) o kibibyte (KiB). 1024 bits se denomina kilobit (Kb o Kbit) o kibibit (Kib o Kibit).

De manera similar, MB/MiB, Mb/Mib, GB/GiB y Gb/Gib se utilizan para millones y billones de bytes y bits. La capacidad de la memoria generalmente se mide en bytes.

La velocidad de comunicación suele medirse en potencias de diez bits/segundo. Por ejemplo, la velocidad máxima de un módem de acceso telefónico suele ser de 56 kbits/seg, que son 56.000 bits/seg.

SUMA BINARIA

La suma binaria es muy parecida a la suma decimal pero más fácil, como se muestra:

$$\begin{array}{r} 11 \quad \leftarrow \text{carries} \rightarrow \\ 4277 \\ + 5499 \\ \hline 9776 \end{array}$$

Como en la suma decimal, si la suma de dos números es mayor que lo que cabe en un solo dígito, llevamos (carry) un 1 a la siguiente columna. En la columna de la derecha de la figura, $7 + 9 = 16$, que no cabe en un solo dígito porque es mayor que 9. Entonces, registramos el dígito de las unidades, 6, y llevamos el dígito de las decenas, 1, a la siguiente columna.

SUMA BINARIA

Del mismo modo, en binario, si la suma de dos números es mayor que 1, trasladamos el dígito 2 a la siguiente columna.

$$\begin{array}{r} \leftarrow \text{carries} \rightarrow \\ 11 \\ 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

Por ejemplo, en la columna más a la derecha, la suma $1 + 1 = 2_{10} = 10_2$ no cabe en un solo dígito binario. Entonces, registramos el dígito (0) y anotamos el carry (1). En la segunda columna, la suma es $1 + 1 + 1 = 3_{10} = 11_2$. Registramos el (1) y carry (1). Por razones obvias, el bit que se transfiere a la columna vecina se denomina carry bit.

SUMA BINARIA - Ejemplo

Calcule $0111_2 + 0101_2$

$$\begin{array}{r} \text{111} \\ 0111 \\ + 0101 \\ \hline 1100 \end{array}$$

muestra que la suma es 1100_2 . Los carry se indican en azul. Podemos comprobar nuestro trabajo repitiendo el cálculo en decimal. $0111_2 = 7_{10}$. $0101_2 = 5_{10}$.

La suma es $12_{10} = 1100_2$.

SUMA BINARIA CON OVERFLOW

Los sistemas digitales suelen funcionar con un número fijo de dígitos.

*Se dice que la suma se desborda (**overflow**) si el resultado es demasiado grande para caber en los dígitos disponibles.*

Un número de 4 bits, por ejemplo, tiene el rango $[0, 15]$. La suma binaria de 4 bits se desborda si el resultado excede 15.

El quinto bit de la suma se descarta, lo que produce un resultado incorrecto en los cuatro bits restantes. El desbordamiento se puede detectar comprobando el acarreo de la columna más importante.

SUMA BINARIA CON OVERFLOW - Ejemplo

Calcule $1101_2 + 0101_2$ ¿Se produce desbordamiento?

$$\begin{array}{r} \textcolor{blue}{11} \textcolor{blue}{1} \\ 1101 \\ + 0101 \\ \hline 10010 \end{array}$$

muestra que la suma es 10010_2 . Este resultado desborda el rango de un número binario de 4 bits. Si se debe almacenar como cuatro bits, el bit más significativo se descarta, dejando el resultado incorrecto de 0010_2 .

Si el cálculo se hubiera hecho usando números con cinco o más bits, el resultado 10010_2 habría sido correcto.

NUMEROS BINARIOS SIGNADOS

Hasta ahora, hemos considerado sólo números binarios sin signo que representan cantidades positivas.

A menudo queremos representar números tanto positivos como negativos, lo que requerirá un sistema numérico binario diferente.

Existen varios esquemas para representar números binarios con signo.

*Los dos más utilizados se denominan **signo/magnitud** y **complemento a dos**.*

NUMEROS SIGNO/MAGNITUD

Los números de signo/magnitud son intuitivamente atractivos porque coinciden con nuestra costumbre de escribir números negativos con un signo menos seguido de la magnitud.

Un número de signo/magnitud de N bits utiliza el bit más significativo como signo y los $N - 1$ bits restantes como magnitud (valor absoluto).

*Un bit de signo de **0 indica positivo** y un bit de signo de **1 indica negativo**.*

NUMEROS SIGNO/MAGNITUD - Ejemplo

Escriba 5 y -5 como números de signo/magnitud de 4 bits.

Ambos números tienen una magnitud de $5_{10} = 101_2$.

Por lo tanto,

$$5_{10} = 0101_2$$

Y

$$-5_{10} = 1101_2$$

NUMEROS SIGNO/MAGNITUD

Desafortunadamente, la suma binaria ordinaria no funciona para números de signo/magnitud.

Por ejemplo, usar la suma ordinaria de $-5_{10} + 5_{10}$ da $1101_2 + 0101_2 = 10010_2$, lo cual no tiene sentido.

Un número de signo/magnitud de N bits abarca el rango

$$[-2^{N-1} + 1, 2^{N-1} - 1].$$

Los números de signo/magnitud son ligeramente impares porque existen tanto $+0$ como -0 . Ambos indican cero. Como es de esperar, puede resultar problemático tener dos representaciones diferentes para el mismo número.

NUMEROS COMPLEMENTO A 2

Los números en complemento a dos son idénticos a los números binarios sin signo, excepto que la posición del bit más significativo tiene un peso de -2^{N-1} en lugar de 2^{N-1} . Superan las deficiencias de los números de signo/magnitud: el cero tiene una representación única y la suma ordinaria funciona.

En la representación en complemento a dos, el cero se escribe todo ceros: $00 \dots 000_2$.

El número más positivo tiene un 0 en la posición más significativa y 1 en el resto: $01 \dots 111_2 = 2^{N-1} - 1$.

El número más negativo tiene un 1 en la posición más significativa y 0 en el resto: $10 \dots 000_2 = -2^{N-1}$. Y -1 se escribe como todos unos: $11 \dots 111_2$.

NUMEROS COMPLEMENTO A 2

Observe que *los números positivos tienen un 0 en la posición más significativa y los números negativos tienen un 1 en esta posición*, por lo que el bit más significativo puede verse como el bit de signo. Sin embargo, el número total se interpreta de manera diferente para los números en complemento a dos y los números de signo/magnitud.

El signo de un número en complemento a dos se invierte (por ejemplo, de +5 a -5 o de -17 a +17) invirtiendo los bits del número y luego sumando 1 a la posición del bit menos significativo. Este proceso se llama método de inversión del signo. Es útil para encontrar la representación de un número negativo o determinar la magnitud de un número negativo.

NUMEROS COMPLEMENTO A 2 - Ejemplo

*Encuentre la representación de -2_{10} como un número en complemento a dos de **4** bits.*

Comience representando la magnitud: $+2_{10} = 0010_2$.

Para obtener -2_{10} , invierta el signo invirtiendo los bits y sumando 1.

Invertir 0010_2 produce 1101_2 .

$$1101_2 + 1 = 1110_2.$$

Entonces, -2_{10} es 1110_2 .

NUMEROS COMPLEMENTO A 2 - Ejemplo

*Encuentre el valor decimal del número en complemento a dos de **4** bits 0x9 (es decir, 1001_2).*

1001_2 tiene un 1 inicial, por lo que debe ser negativo.

Para encontrar su magnitud, invierta el signo invirtiendo los bits y sumando 1.

Invirtiendo $1001_2 = 0110_2$. Sumando uno: $0110_2 + 1 = 0111_2 = 7_{10}$.

Por lo tanto, $1001_2 = -7_{10}$.

NUMEROS COMPLEMENTO A 2 - Ejemplo

Encuentre el valor decimal del número en complemento a dos de **4** bits 0x9 (es decir, 1001_2).

1001_2 tiene un 1 inicial, por lo que debe ser negativo.

Para encontrar su magnitud, invierta el signo invirtiendo los bits y sumando 1.

Invirtiendo $1001_2 = 0110_2$. Sumando uno: $0110_2 + 1 = 0111_2 = 7_{10}$.

Por lo tanto, $1001_2 = -7_{10}$.

NUMEROS COMPLEMENTO A 2 SUMA

Los números en complemento a dos tienen la ventaja indiscutible de que la suma funciona correctamente tanto para números positivos como negativos.

Recordemos que al sumar números de N bits, el acarreo del bit N (es decir, el bit del resultado $N + 1$) se descarta.

NUMEROS COMPLEMENTO A 2 SUMA

Los números en complemento a dos tienen la ventaja indiscutible de que la suma funciona correctamente tanto para números positivos como negativos.

Recordemos que al sumar números de N bits, el acarreo del bit N (es decir, el bit del resultado $N + 1$) se descarta.

NUMEROS COMPLEMENTO A 2 SUMA - Ejemplo

Calcula (a) $-2_{10} + 1_{10}$ y (b) $-7_{10} + 7_{10}$ usando números en complemento a dos.

$$(a) -2_{10} + 1_{10} = 1110_2 + 0001_2 = 1111_2 = -1_{10}.$$

$$(b) -7_{10} + 7_{10} = 1001_2 + 0111_2 = 10000_2.$$

El quinto bit se descarta, obteniéndose el resultado correcto de 4 bits: 0000_2 .

NUMEROS COMPLEMENTO A 2 RESTA

La resta se realiza tomando el complemento a dos del segundo número y luego sumándolo. Calcula (a) $5_{10} - 3_{10}$ y (b) $3_{10} - 5_{10}$ usando números de complemento a dos de 4 bits.

(a) $3_{10} = 0011_2$. Tomamos su complemento a dos para obtener $-3_{10} = 1101_2$. Ahora, sumamos $5_{10} + (-3_{10}) = 0101_2 + 1101_2 = 0010_2 = 2_{10}$. Nótese que el acarreo de la posición más significativa se descarta porque el resultado se almacena en cuatro bits.

(b) Tomamos el complemento a dos de 5_{10} para obtener $-5_{10} = 1011$. Ahora, sumamos $3_{10} + (-5_{10}) = 0011_2 + 1011_2 = 1110_2 = -2_{10}$.

NUMEROS COMPLEMENTO A 2 DE CERO

Al igual que los números sin signo, los números en complemento a dos de N bits representan uno de los 2^N valores posibles.

Sin embargo, estos valores se dividen entre positivos y negativos.

Por ejemplo, un número sin signo de 4 bits representa 16 valores: del 0 al 15.

Un número en complemento a dos de 4 bits también representa 16 valores: del -8 al 7.

En general, el rango de un número en complemento a dos de N bits abarca $[-2^{N-1}, 2^{N-1} - 1]$.

NUMEROS COMPLEMENTO A 2 DE CERO

Es lógico que haya un número negativo más que positivo, ya que no existe el -0.

*El número más negativo, $10 \dots 000_2 = -2^{N-1}$, a veces se denomina número extraño (**weird number**).*

Su complemento a dos se obtiene invirtiendo los bits (lo que produce $01 \dots 111_2$) y sumando 1, lo que produce $10 \dots 000_2$, el número extraño, de nuevo.

Por lo tanto, este número negativo no tiene contraparte positiva.

NUMEROS COMPLEMENTO A 2 DE CERO

La suma de dos números positivos o negativos de N bits puede provocar un desbordamiento si el resultado es mayor que $2^{N-1} - 1$ o menor que -2^{N-1} .

La suma de un número positivo a uno negativo nunca provoca un desbordamiento.

A diferencia de los números sin signo, un acarreo de la columna más significativa no indica desbordamiento.

En cambio, el desbordamiento se produce si los dos números que se suman tienen el mismo bit de signo y el resultado tiene el bit de signo opuesto.

NUMEROS COMPLEMENTO A 2 CON OVERFLOW - Ejemplo

Calcula $4_{10} + 5_{10}$ usando números en complemento a dos de 4 bits. ¿El resultado se desborda?

$$4_{10} + 5_{10} = 0100_2 + 0101_2 = 1001_2 = -7_{10}$$

El resultado excede el rango de números positivos en complemento a dos de 4 bits, lo que produce un resultado negativo incorrecto.

Si el cálculo se hubiera realizado con cinco o más bits, el resultado $01001_2 = 9_{10}$ habría sido correcto.

NUMEROS COMPLEMENTO A 2 CON OVERFLOW - Ejemplo

Cuando un número en complemento a dos se extiende a más bits, el bit de signo debe copiarse en las posiciones de bits más significativas.

*Este proceso se denomina extensión de signo (**sign extension**).*

*Por ejemplo, los números 3 y -3 se escriben como números en complemento a dos de **4** bits: 0011 y 1101, respectivamente.*

*Se extienden a siete bits copiando el bit de signo en los tres bits superiores para formar **000**0011 y **111**101, respectivamente, que son representaciones de 7 bits de 3 y -3.*

COMPARACIÓN DE SISTEMAS NUMÉRICOS

Los tres sistemas de numeración binaria más comunes son el binario sin signo, el de complemento a dos y el de signo/magnitud. La Tabla compara el rango de números de N bits en cada uno de estos tres sistemas.

System	Range
Unsigned	$[0, 2^N - 1]$
Two's Complement	$[-2^{N-1}, 2^{N-1} - 1]$
Sign/Magnitude	$[-2^{N-1} + 1, 2^{N-1} - 1]$

COMPARACIÓN DE SISTEMAS NUMÉRICOS

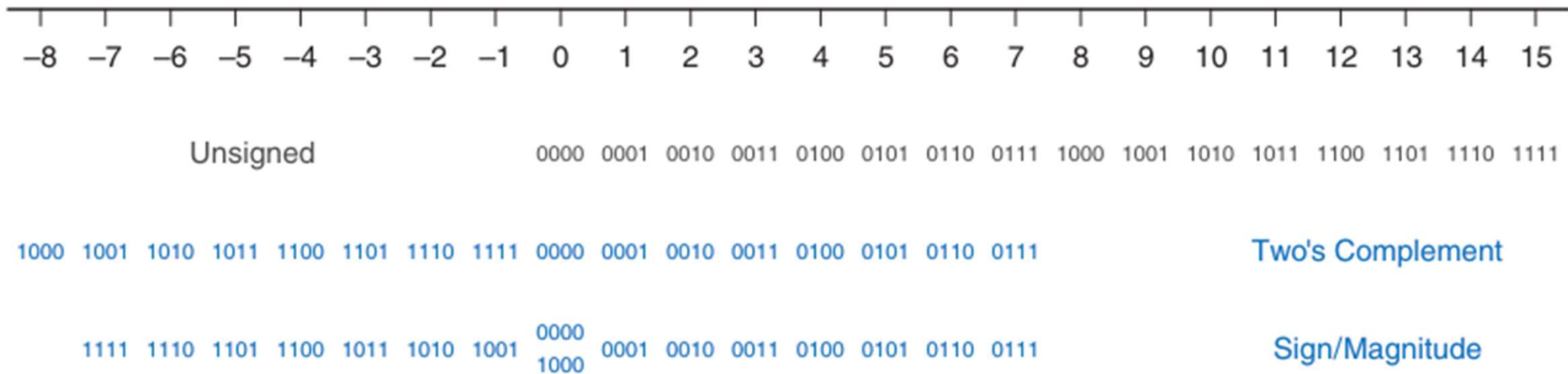
Los números en complemento a dos son convenientes porque representan tanto enteros positivos como negativos y porque la suma ordinaria funciona para todos los números.

La resta se realiza negando el segundo número (es decir, invirtiendo el signo) y luego sumándolo.

Salvo que se indique lo contrario, se asume que todos los números binarios con signo utilizan la representación en complemento a dos.

COMPARACIÓN DE SISTEMAS NUMÉRICOS

La figura muestra una recta numérica que indica los valores de los números de 4 bits en cada sistema.



COMPARACIÓN DE SISTEMAS NUMÉRICOS

Los números sin signo abarcan el intervalo $[0, 15]$ en orden binario regular.

Los números en complemento a dos abarcan el intervalo $[-8, 7]$.

Los números no negativos $[0, 7]$ comparten la misma codificación que los números sin signo.

Los números negativos $[-8, -1]$ se codifican de tal manera que un valor binario sin signo mayor representa un número más cercano a 0.

Nótese que el número 1000 representa -8 y no tiene contraparte positiva.

COMPARACIÓN DE SISTEMAS NUMÉRICOS

Los números con signo/magnitud abarcan el intervalo $[-7, 7]$. El bit más significativo es el bit de signo.

Los números positivos $[1, 7]$ comparten la misma codificación que los números sin signo. Los números negativos son simétricos, pero tienen el bit de signo activado.

El 0 se representa tanto con 0000 como con 1000. Por lo tanto, los números de signo/magnitud de N bits representan solo $2^N - 1$ enteros debido a las dos representaciones del 0.

Los números binarios sin signo o en complemento a dos de N bits representan 2^N enteros.

SISTEMAS NUMÈRICOS

Ing. Edgar Andrés Gutiérrez Cáceres